We analyzed the same Python project (src/tokenizer.py, src/parser.py, src/upload.py) using the same LLM twice.

Aspect	LLM 1 *(Source: llm_quality_check.txt)*	LLM 2 *(Source: llm_quality_check2.txt)*
Overall tone	Technical and structured, resembling a static analyzer report.	Conversational and thorough, like a peer review by a senior engineer.
Main focus	Syntax correctness, potential runtime errors, and style.	Correctness, robustness, test coverage, and maintainability.
Severity classification	No severity labels.	Categorized findings as *High*, *Medium*, and *Low* severity for prioritization.
Tokenizer issues	Reported three main issues: • Non-relative import (`from upload import FileLoader`). • Use of `strip()` affecting line numbering. • Enum formatting bug in `print_tokens`.	Confirmed the same three issues but added deeper insights: risk of circular import and better explanation of how `splitlines()` preserves structure.
Parser issues	Highlighted `_peek` returning the last token on end-of-stream, `_advance` potential `IndexError`, and the need for error tests.	Expanded analysis with detailed reasoning: infinite-loop risk, unsafe `replace('if','')` keyword stripping, and lack of explicit error handling for unmatched END tokens.
Upload module issues	Suggested opening files with explicit encoding (`utf-8`) and validating file type.	Reiterated this but explained cross-platform decoding risks and recommended adding error handling for unreadable files.
Testing recommendations	Proposed adding parser error-path tests and tokenizer edge cases.	Much more comprehensive: specific unit test suggestions for empty input, unmatched END tokens, nested blocks, and keyword-in-value cases.

Examples / Fixes	Included sample fixes for formatting and file reading.	Included multiple detailed “patch suggestions” and next-step plans for each module.